



VSM'S SOMSHEKAR R KOTHIWALE INSTITUTE OF TECHNOLOGY NIPANI 591237

COMPUTER GRAPHICS AND IMAGE PROCESSING LABORATORY

Course Code	21CSL66	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0:0:2:0	SEE Marks	50
Total Hours of Pedagogy	24	Total Marks	100
Credits	1	Exam Hours	03

Course Objectives:

CLO 1: Demonstrate the use of Open GL.

CLO 2: Demonstrate the different geometric object drawing

using OpenGL
CLO 3: Demonstration of 2D/3D transformation on simple objects.

CLO 4: Demonstration of lighting effects on the created objects.

CLO 5: Demonstration of Image processing operations on image/s.

Sl. No.	Practice Programs
	Installation of OpenGL /OpenCV/ Python and required headers Simple programs using OpenGL (Drawing simple geometric object like line, circle, rectangle, square) Simple programs using OpenCV (operation on an image/s)
	PART A
	<i>List of problems for which student should develop program and execute in the Laboratory using OpenGL/openCV/ Python</i>
1.	Develop a program to draw a line using Bresenham's line drawing technique
2.	Develop a program to demonstrate basic geometric operations on the 2D object
3.	Develop a program to demonstrate basic geometric operations on the 3D object
4.	Develop a program to demonstrate 2D transformation on basic objects
5.	Develop a program to demonstrate 3D transformation on 3D objects
6.	Develop a program to demonstrate Animation effects on simple objects.
7.	Write a Program to read a digital image. Split and display image into 4 quadrants, up, down, right and left.
8.	Write a program to show rotation, scaling, and translation on an image.
9.	Read an image and extract and display low-level features such as edges, textures using filtering techniques.
10.	Write a program to blur and smoothing an image.
11.	Write a program to contour an image.
12.	Write a program to detect a face/s in an image.

1. Develop a program to draw a line using Bresenham's line drawing technique.

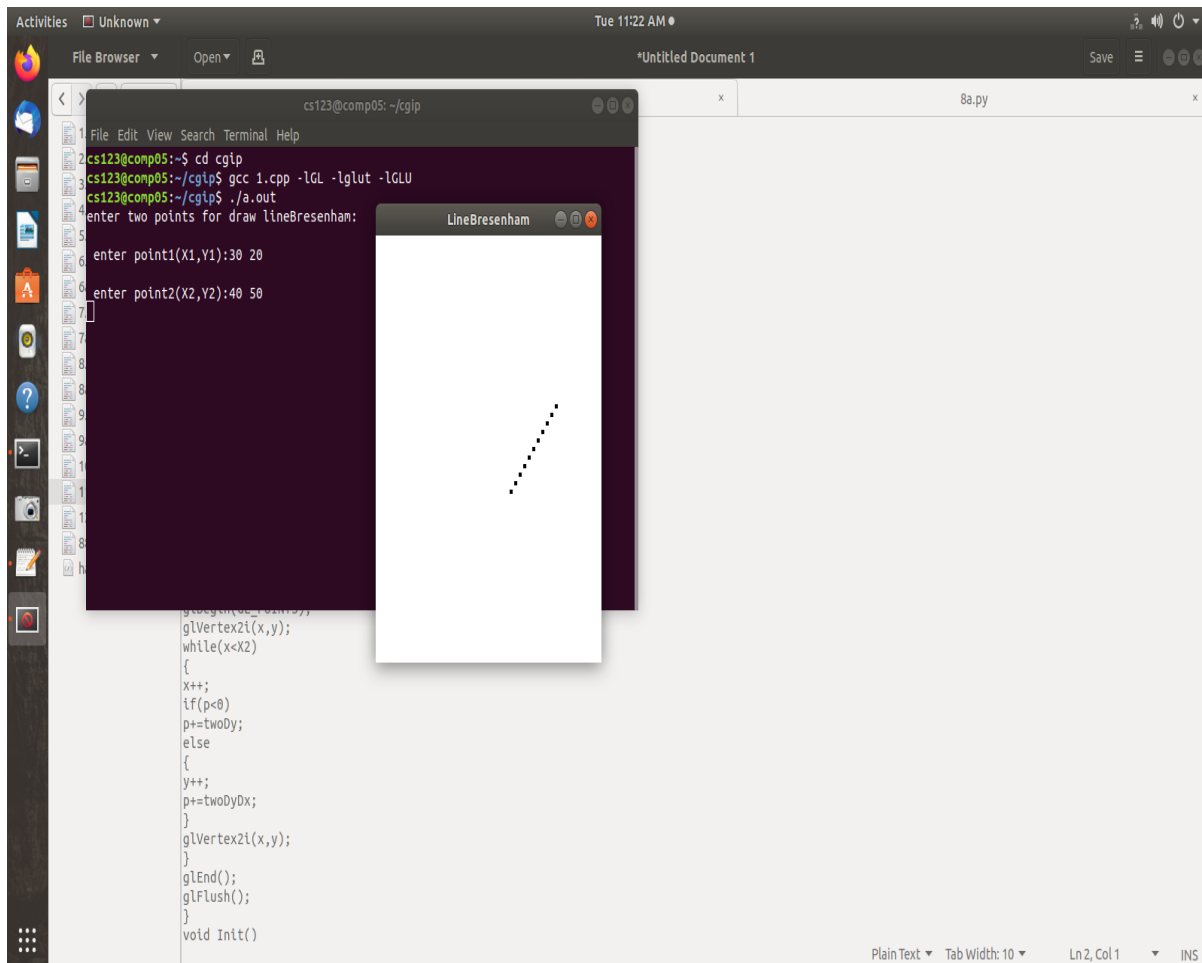
```
#include<stdio.h>
#include<math.h>
#include<GL/glut.h>
GLint X1,Y1,X2,Y2;
void LineBres(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    int dx=abs(X2-X1),dy=abs(Y2-Y1);
    int p=2*dy-dx;
    int twoDy=2*dy, twoDyDx=2*(dy-dx);
    int x,y;
    if(X1>X2)
    {
        x=X2;
        y=Y2;
        X2=X1;
    }
    else
    {
        x=X1;
        y=Y1;
        X2=X2;
    }
    glBegin(GL_POINTS);
    glVertex2i(x,y);
    while(x<X2)
    {
        x++;
        if(p<0)
            p+=twoDy;
        else
        {
            y++;
            p+=twoDyDx;
        }
        glVertex2i(x,y);
    }
    glEnd();
    glFlush();
}
void Init()
{
    glClearColor(1.0,1.0,1.0,0);
    glColor3f(0.0,0.0,0.0);
    glPointSize(4.0);
    glViewport(0,0,50,50);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0,50,0,50);
}
int main(int argc,char **argv)
{
    printf("enter two points for draw lineBresenham:\n");
```

```

printf("\n enter point1(X1,Y1):");
scanf("%d%d",&X1,&Y1);
printf("\n enter point2(X2,Y2):");
scanf("%d%d",&X2,&Y2);
glutInit(&argc,argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(300,400);
glutInitWindowPosition(0,0);
glutCreateWindow("LineBresenham");
Init();
glutDisplayFunc(LineBres);
glutMainLoop();
}

```

Output:



2. Develop a program to demonstrate basic geometric operations on the 2D object

```

#include <stdio.h>
#include <GL/glut.h>
typedef float point2[2];

```

```

/* initial triangle */

point2 v[]={ {-1.0, -0.58}, {1.0, -0.58}, {0.0, 1.15} };
int n;

/* display one triangle */
void triangle( point2 a, point2 b, point2 c)
{
    glBegin(GL_TRIANGLES);
        glVertex2fv(a);
        glVertex2fv(b);
        glVertex2fv(c);
    glEnd();
}

void divide_triangle(point2 a, point2 b, point2 c, int m)
{
    /* triangle subdivision using vertex numbers */

    point2 v0, v1, v2;
    int j;
    if(m>0)
    {
        for(j=0; j<2; j++) v0[j]=(a[j]+b[j])/2;
        for(j=0; j<2; j++) v1[j]=(a[j]+c[j])/2;
        for(j=0; j<2; j++) v2[j]=(b[j]+c[j])/2;
        divide_triangle(a, v0, v1, m-1);
        divide_triangle(c, v1, v2, m-1);
        divide_triangle(b, v2, v0, m-1);
    }
    else(triangle(a,b,c)); /* draw triangle at end of recursion */
}

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);a
    divide_triangle(v[0], v[1], v[2], n);
    glFlush();
}

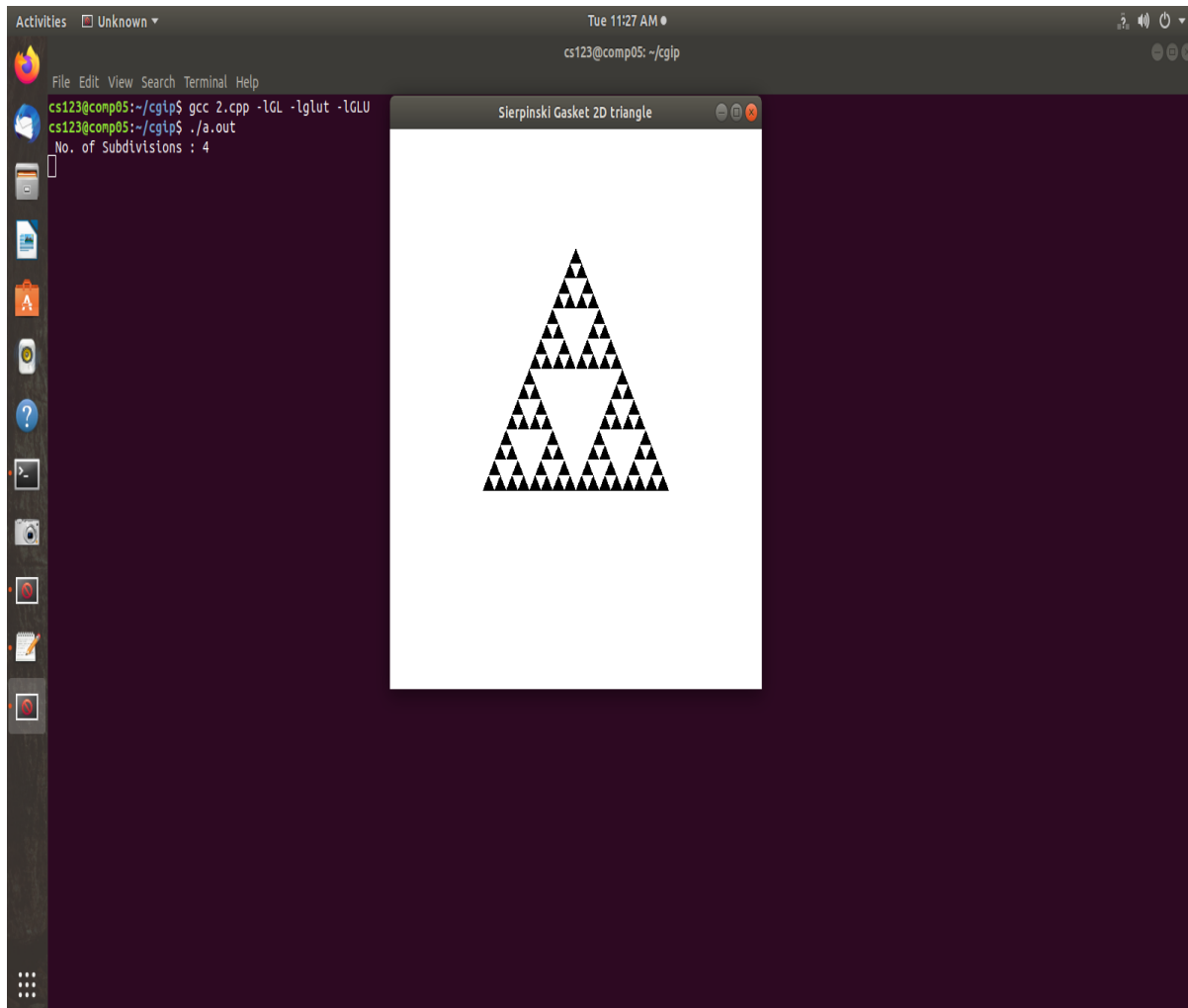
void myinit()
{
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-2.0, 2.0, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glColor3f(0.0,0.0,0.0);
}

int main(int argc, char **argv)
{
    printf(" No. of Subdivisions : ");
    scanf("%d",&n);
    glutInit(&argc, argv);

```

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
glutInitWindowSize(500, 500);
glutCreateWindow("Sierpinski Gasket 2D triangle");
glutDisplayFunc(display);
    myinit();
glutMainLoop();
}
```

Output:



3. Develop a program to demonstrate basic geometric operations on the 3D object

```
#include <stdio.h>
#include <stdlib.h>
#include <GL/glut.h>
typedef float point[3];

/* initial tetrahedron */
point v[]={ {0.0, 0.0, 1.0}, {0.0, 0.942809, -0.33333},
            {-0.816497, -0.471405, -0.333333}, {0.816497, -0.471405, -0.333333} };
static GLfloat theta[] = {0.0,0.0,0.0};
int n;
void triangle( point a, point b, point c)
/* display one triangle using a line loop for wire frame, a single
normal for constant shading, or three normals for interpolative shading */
{
    glBegin(GL_POLYGON);
    glNormal3fv(a);
    glVertex3fv(a);
    glVertex3fv(b);
    glVertex3fv(c);
    glEnd();
}
/* triangle subdivision using vertex numbers
righthand rule applied to create outward pointing faces */
void divide_triangle(point a, point b, point c, int m)
{
    point v1, v2, v3;
    int j;
    if(m>0)
    {
        for(j=0; j<3; j++) v1[j]=(a[j]+b[j])/2;
        for(j=0; j<3; j++) v2[j]=(a[j]+c[j])/2;
        for(j=0; j<3; j++) v3[j]=(b[j]+c[j])/2;
        divide_triangle(a, v1, v2, m-1);
        divide_triangle(c, v2, v3, m-1);
        divide_triangle(b, v3, v1, m-1);
    }
    else(triangle(a,b,c)); /* draw triangle at end of recursion */
}
/* Apply triangle subdivision to faces of tetrahedron */
void tetrahedron( int m)
{
    glColor3f(1.0,0.0,0.0);
    divide_triangle(v[0], v[1], v[2], m);
    glColor3f(0.0,1.0,0.0);
    divide_triangle(v[3], v[2], v[1], m);
    glColor3f(0.0,0.0,1.0);
    divide_triangle(v[0], v[3], v[1], m);
    glColor3f(0.0,0.0,0.0);
    divide_triangle(v[0], v[2], v[3], m);
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    tetrahedron(3);
}
```

```

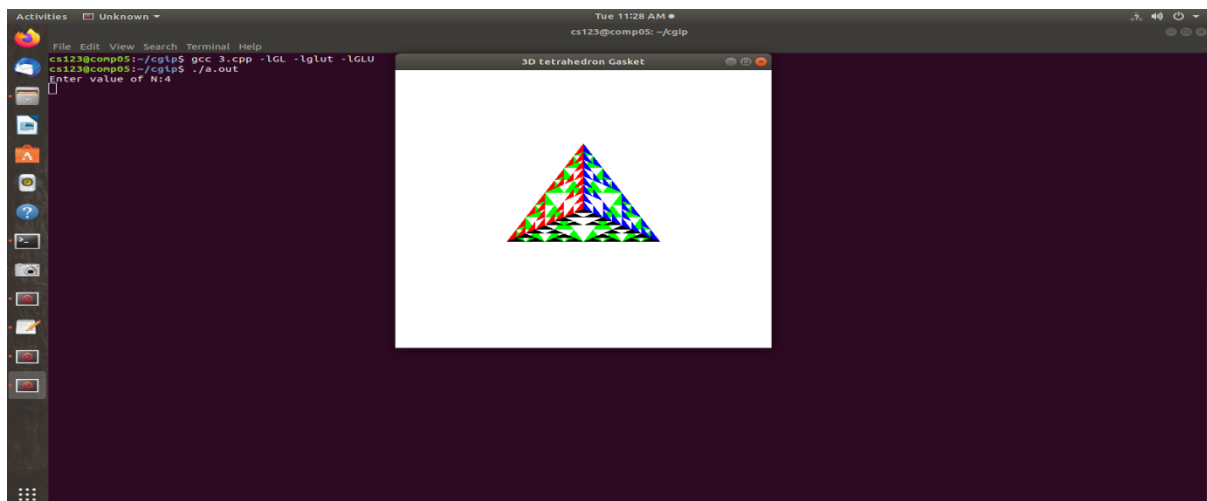
    glFlush();
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
                2.0 * (GLfloat) h / (GLfloat) w, -10.0, 10.0);
    else
        glOrtho(-2.0 * (GLfloat) w / (GLfloat) h,
                2.0 * (GLfloat) w / (GLfloat) h, -2.0, 2.0, -10.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
    glutPostRedisplay();
}

int main(int argc, char **argv)
{
    int i = 0;
    printf("Enter value of N:");
    scanf("%d", &i);
    n = i;
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("3D tetrahedron Gasket");
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glEnable(GL_DEPTH_TEST);
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glutMainLoop();
}

```

Output:



4. Develop a program to demonstrate 2D transformation on basic objects.

```

#include<GL/glut.h>
#include<stdio.h>

```

```

int x,y;
int where_to_rotate=0;
float rotate_angle=0;
float translate_x=0,translate_y=0;
void draw_pixel(float x1, float y1)
{
    glPointSize(5);
    glBegin(GL_POINTS);
        glVertex2f(x1,y1);    // plot a single point
    glEnd();
}
void triangle(int x, int y)
{
    glColor3f(1,0,0);
    glBegin(GL_POLYGON);    // drawing a Triangle
        glVertex2f(x,y);
        glVertex2f(x+400,y+300);
        glVertex2f(x+300,y+0);
    glEnd();
}
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();
    glColor3f(1,1,1);
    draw_pixel(0,0);
    if (where_to_rotate == 1)
    {
        translate_x = 0;
        translate_y = 0;
        rotate_angle += 1;
    }
    if (where_to_rotate == 2)
    {
        translate_x = x;
        translate_y = y;
        rotate_angle += 1;
        glColor3f(0,0,1);
        draw_pixel(x,y);
    }
    glTranslatef(translate_x, translate_y, 0);
    glRotatef(rotate_angle, 0, 0, 1);
    glTranslatef(-translate_x, -translate_y, 0);
    triangle(translate_x,translate_y);
    glutPostRedisplay();
    glutSwapBuffers();
}
void init()
{
    glClearColor(0,0,0,1); //setting to black
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-800, 800, -800, 800);
    glMatrixMode(GL_MODELVIEW);
}
void rotateMenu (int option)
{
    if(option==1)

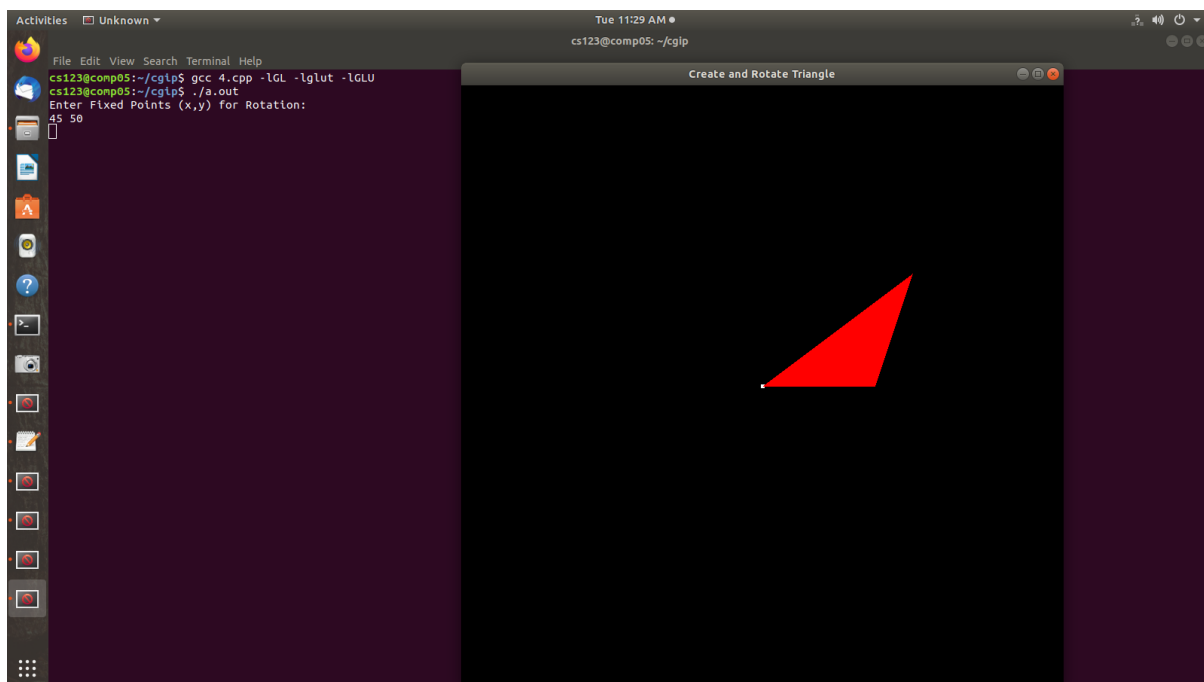
```

```

where_to_rotate=1;
if(option==2)
where_to_rotate=2;
if(option==3)
where_to_rotate=3;
}
int main(int argc, char **argv)
{
printf( "Enter Fixed Points (x,y) for Rotation: \n");
scanf("%d %d", &x, &y);
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE|GLUT_RGB);
glutInitWindowSize(800, 800);
glutInitWindowPosition(0, 0);
glutCreateWindow("Create and Rotate Triangle");
init();
glutDisplayFunc(display);
glutCreateMenu(rotateMenu);
glutAddMenuEntry("Rotate around ORIGIN",1);
glutAddMenuEntry("Rotate around FIXED POINT",2);
glutAddMenuEntry("Stop Rotation",3);
glutAttachMenu(GLUT_RIGHT_BUTTON);
glutMainLoop();
}

```

Output:



5. Develop a program to demonstrate 3D transformation on 3D objects.

```

#include <stdlib.h>
#include <GL/glut.h>
GLfloat vertices[][3] = {{-1.0,-1.0,-1.0},{1.0,-1.0,-1.0},
                        {1.0,1.0,-1.0}, {-1.0,1.0,-1.0}, {-1.0,-1.0,1.0},
                        {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}};

GLfloat colors[][3] = {{0.0,0.0,0.0},{1.0,0.0,0.0},

```

```

        {1.0,1.0,0.0}, {0.0,1.0,0.0}, {0.0,0.0,1.0},
        {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}};

void polygon(int a, int b, int c, int d)
{
    glBegin(GL_POLYGON);
        glColor3fv(colors[a]);
        glVertex3fv(vertices[a]);
        glColor3fv(colors[b]);
        glVertex3fv(vertices[b]);
        glColor3fv(colors[c]);
        glVertex3fv(vertices[c]);
        glColor3fv(colors[d]);
        glVertex3fv(vertices[d]);
    glEnd();
}

void colorcube()
{
    polygon(0,3,2,1);
    polygon(2,3,7,6);
    polygon(0,4,7,3);
    polygon(1,2,6,5);
    polygon(4,5,6,7);
    polygon(0,1,5,4);
}

static GLfloat theta[] = {0.0,0.0,0.0};
static GLint axis = 2;
static GLdouble viewer[] = {0.0, 0.0, 5.0}; /* initial viewer location */

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    /* Update viewer position in modelview matrix */
    glLoadIdentity();
    gluLookAt(viewer[0],viewer[1],viewer[2], 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
    /* rotate cube */
    glRotatef(theta[0], 1.0, 0.0, 0.0);
    glRotatef(theta[1], 0.0, 1.0, 0.0);
    glRotatef(theta[2], 0.0, 0.0, 1.0);
    colorcube(); /* draw the rotated color cube */
    glFlush();
    glutSwapBuffers();
}

void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0;
    if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1; if(btn==GLUT_RIGHT_BUTTON
    && state == GLUT_DOWN) axis = 2;
    theta[axis] += 4.0;
    if( theta[axis] > 360.0 ) theta[axis] -= 360.0;
    display();
}

void keys(unsigned char key, int x, int y)
{

```

```

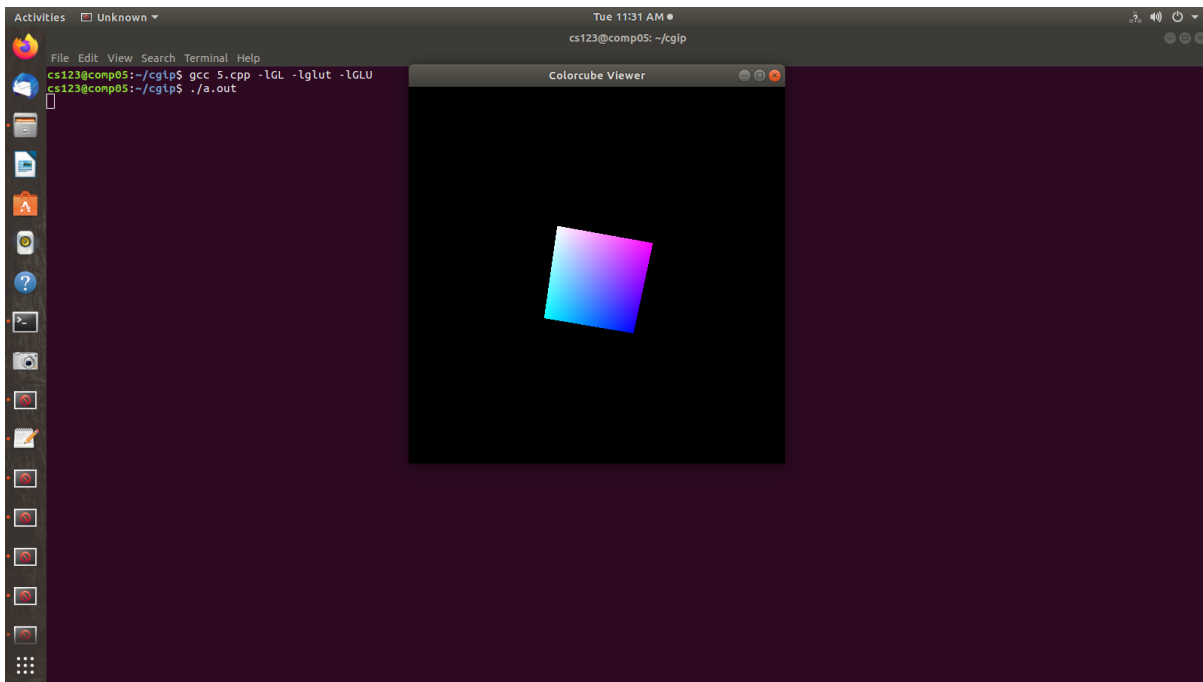
/* Use x, X, y, Y, z, and Z keys to move viewer */
if(key == 'x') viewer[0]-= 1.0;
if(key == 'X') viewer[0]+= 1.0;
if(key == 'y') viewer[1]-= 1.0;
if(key == 'Y') viewer[1]+= 1.0;
if(key == 'z') viewer[2]-= 1.0;
if(key == 'Z') viewer[2]+= 1.0;
display();
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    /* Use a perspective view */
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if(w<=h) glFrustum(-2.0, 2.0, -2.0 * (GLfloat) h/ (GLfloat) w,
        2.0* (GLfloat) h / (GLfloat) w, 2.0, 20.0);
    else glFrustum(-2.0, 2.0, -2.0 * (GLfloat) w/ (GLfloat) h,
        2.0* (GLfloat) w / (GLfloat) h, 2.0, 20.0);
    /* Or we can use gluPerspective that is  gluPerspective(45.0, w/h, -10.0, 10.0); */
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Colorcube Viewer");
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutKeyboardFunc(keys);
    glEnable(GL_DEPTH_TEST);
    glutMainLoop();
}

```

Output:



6. Develop a program to demonstrate Animation effects on simple objects.

```
#include<GL/glut.h>

#include<stdio.h>

#include<math.h>

#define PI 3.1416

static int win,val=0,CMenu;

void CreateMenu(void);
void Menu(int value);

struct wcPt3D
{
    GLfloat x, y, z;
};

GLsizei winWidth = 600, winHeight = 600;
GLfloat xwcMin = 0.0, xwcMax = 130.0;
GLfloat ywcMin = 0.0, ywcMax = 130.0;
void bino(GLint n, GLint *C)
{
    GLint k, j;
    for(k=0;k<=n;k++)
    {
        C[k]=1;
        for(j=n;j>=k+1; j--)

            C[k]*=j;
        for(j=n-k;j>=2;j--)
            C[k]/=j;
    }
}

void computeBezPt(GLfloat u,struct wcPt3D *bezPt, GLint nCtrlPts,struct wcPt3D *ctrlPts,
    GLint *C)
{
    GLint k, n=nCtrlPts-1;
    GLfloat bezBlendFcn;
    bezPt ->x =bezPt ->y = bezPt->z=0.0;
    for(k=0; k< nCtrlPts; k++)
    {
        bezBlendFcn = C[k] * pow(u, k) * pow( 1-u, n-k); bezPt ->x += ctrlPts[k].x * bezBlendFcn; bezPt ->y +=
        ctrlPts[k].y * bezBlendFcn;
        bezPt ->z += ctrlPts[k].z * bezBlendFcn;
    }
}

void bezier(struct wcPt3D *ctrlPts, GLint nCtrlPts, GLint nBezCurvePts)
{
    struct wcPt3D bezCurvePt;
    GLfloat u;
    GLint *C, k;
    C= new GLint[nCtrlPts];
```

```

bino(nCtrlPts-1, C);

glBegin(GL_LINE_STRIP);
for(k=0; k<=nBezCurvePts; k++)

{
u=GLfloat(k)/GLfloat(nBezCurvePts);
computeBezPt(u, &bezCurvePt, nCtrlPts, ctrlPts, C);
glVertex2f(bezCurvePt.x, bezCurvePt.y);
}
glEnd();
delete[]C;
}

void displayFcn()
{
GLint nCtrlPts = 4, nBezCurvePts =20;

static float theta = 0;
struct wcPt3D ctrlPts[4] = {{20, 100, 0},{30, 110, 0},{50, 90, 0},{60, 100, 0}};
ctrlPts[1].x +=10*sin(theta * PI/180.0);
ctrlPts[1].y +=5*sin(theta * PI/180.0);
ctrlPts[2].x -= 10*sin((theta+30) * PI/180.0);
ctrlPts[2].y -= 10*sin((theta+30) * PI/180.0);
ctrlPts[3].x-= 4*sin((theta) * PI/180.0);
ctrlPts[3].y += sin((theta-30) * PI/180.0);
theta+=0.1;
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0, 1.0, 1.0);

glPointSize(5);
//Indian Flag
if(val==1){
glPushMatrix();
glLineWidth(5);
glColor3f(1.0,0.5,0); //Indian flag: Orange color code
for(int i=0;i<8;i++)
{
glTranslatef(0, -0.8, 0);
bezier(ctrlPts, nCtrlPts, nBezCurvePts);
}
glColor3f(1,1,1); //Indian flag: white color code
for(int i=0;i<8;i++)
{
glTranslatef(0, -0.8, 0);
bezier(ctrlPts, nCtrlPts, nBezCurvePts);
}
glColor3f(0,1.0,0); //Indian flag: green color code
for(int i=0;i<8;i++)
{
glTranslatef(0, -0.8, 0);
bezier(ctrlPts, nCtrlPts, nBezCurvePts);
}
glPopMatrix();
glColor3f(0.7, 0.5,0.3);
glLineWidth(5);

glBegin(GL_LINES);
glVertex2f(20,100);

```

```

glVertex2f(20,40);
glEnd();
glFlush();
}
//Karnataka Flag
if(val==2){
glPushMatrix();
glLineWidth(5);
glColor3f(1.0, 1.0, 0.0); //Karnataka flag: Yellow color code
for(int i=0;i<12;i++)
{

glTranslatef(0, -0.8, 0);
bezier(ctrlPts, nCtrlPts, nBezCurvePts);
}
glColor3f(1, 0.0, 0.0); //Karnataka flag: Red color code
for(int i=0;i<12;i++)
{
glTranslatef(0, -0.8, 0);
bezier(ctrlPts, nCtrlPts, nBezCurvePts);
}
glPopMatrix();
glColor3f(0.7, 0.5,0.3);

glLineWidth(5);
glBegin(GL_LINES);
glVertex2f(20,100);
glVertex2f(20,40);
glEnd();
glFlush();
}
glutPostRedisplay();
glutSwapBuffers();
}

void winReshapeFun(GLint newWidth, GLint newHeight)
{
glViewport(0, 0, newWidth, newHeight);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(xwcMin, xwcMax, ywcMin, ywcMax); glClear(GL_COLOR_BUFFER_BIT); }

void CreateMenu(void)
{
CMenu= glutCreateMenu(Menu);//Creaate Menu Option

glutAddMenuEntry("Indian Flag",1);
glutAddMenuEntry("Karnataka Flag",2);

glutAddMenuEntry("Exit",0);

glutAttachMenu(GLUT_RIGHT_BUTTON);

}

void Menu(int value)

```

```

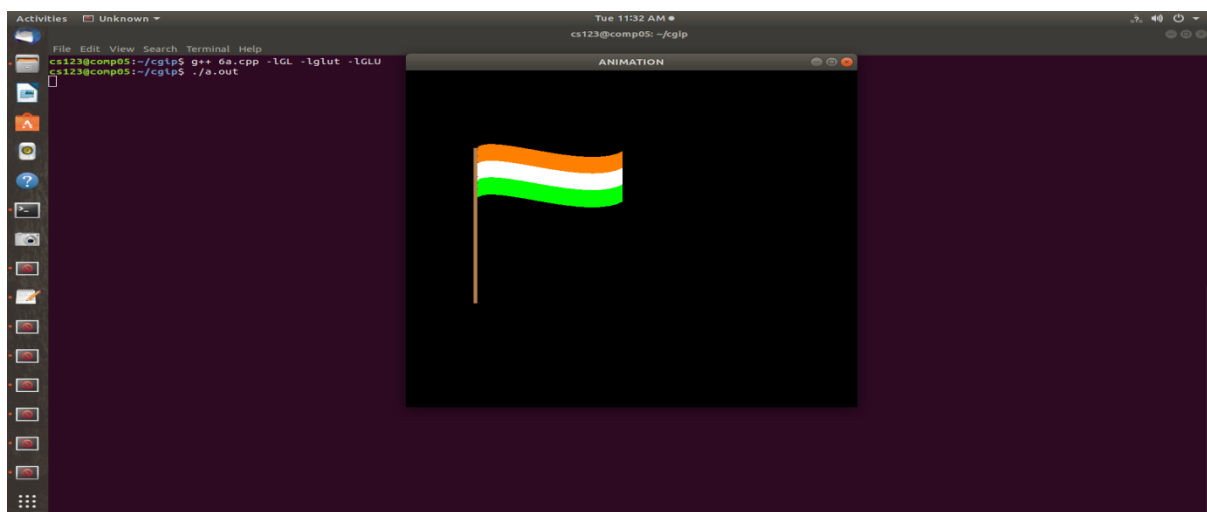
{
if(value==0)
{
glutDestroyWindow(win);
exit(0);
}

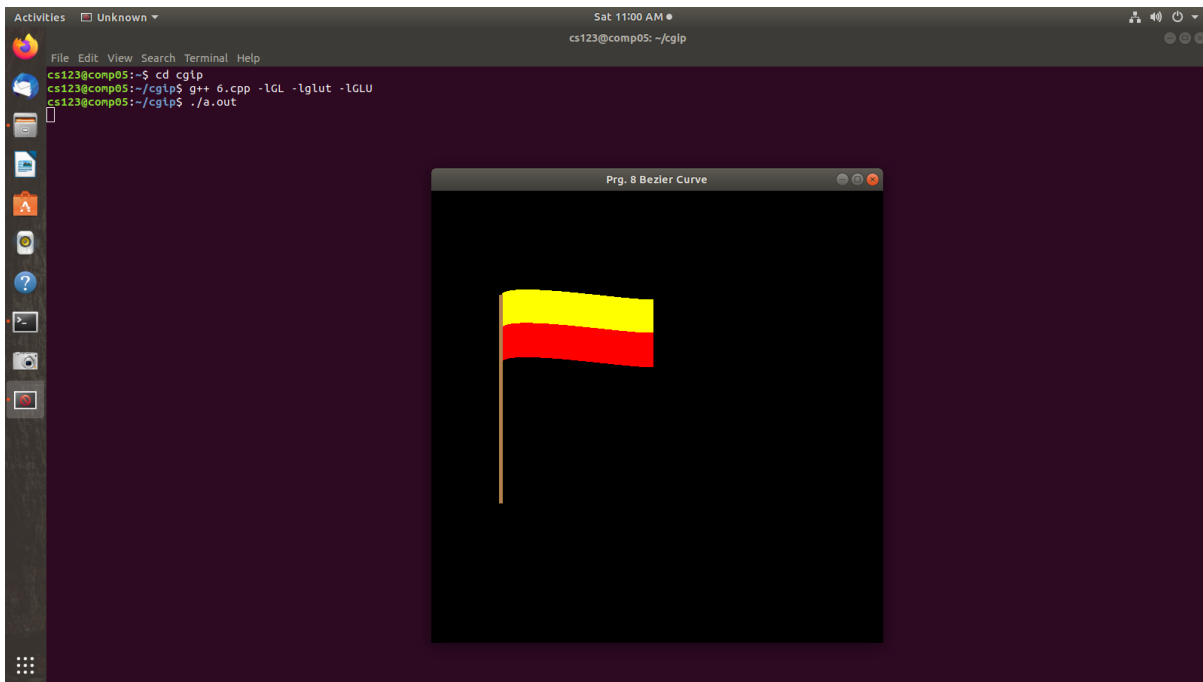
else {
val=value;
}
}

int main(int argc, char **argv)
{
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
glutInitWindowPosition(50, 50);
glutInitWindowSize(winWidth, winHeight);
glutCreateWindow("ANIMATION");
CreateMenu();
glutDisplayFunc(displayFcn);
glutReshapeFunc(winReshapeFun);
glutMainLoop();
}

```

Output:





7. Write a Program to read a digital image. Split and display image into 4 quadrants, up, down, right and left.

```
import cv2

# Load the image
image = cv2.imread('img.jpg')

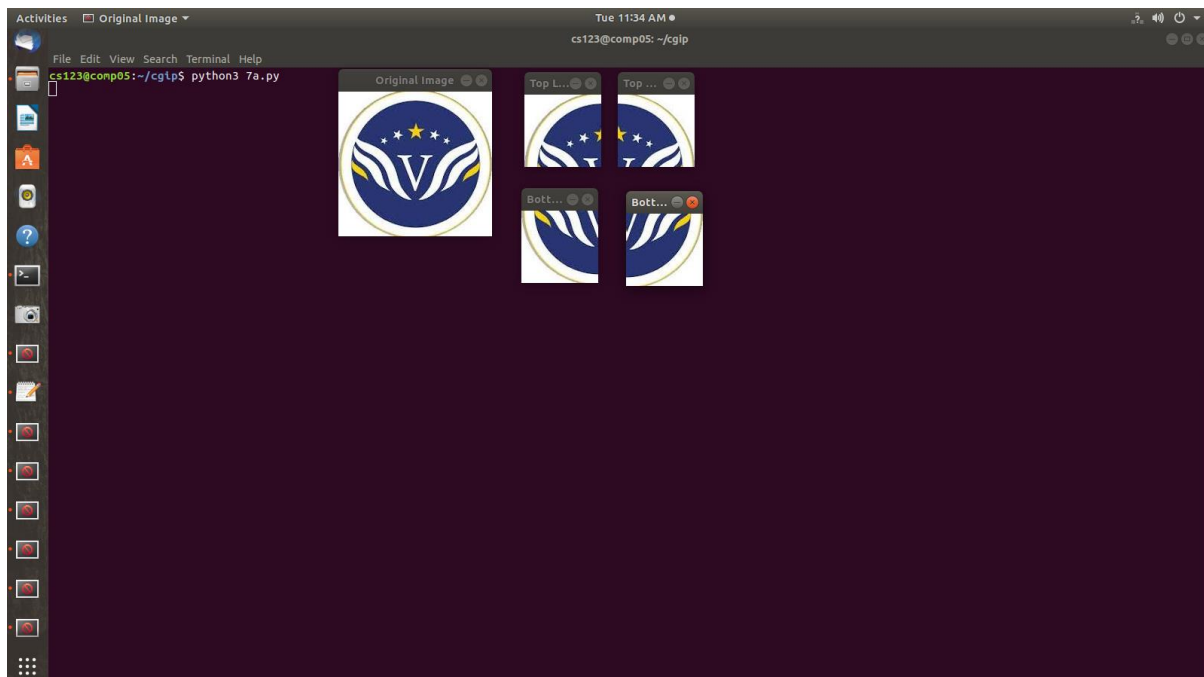
# Get image height and width
height, width = image.shape[:2]

# Split the image into four quadrants
top_left = image[0:height//2, 0:width//2]
top_right = image[0:height//2, width//2:width]
bottom_left = image[height//2:height, 0:width//2]
bottom_right = image[height//2:height, width//2:width]

# Display the original image and the quadrants
cv2.imshow('Original Image', image)
cv2.imshow('Top Left Quadrant', top_left)
cv2.imshow('Top Right Quadrant', top_right)
cv2.imshow('Bottom Left Quadrant', bottom_left)
cv2.imshow('Bottom Right Quadrant', bottom_right)

# Wait for a key press and close all windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :



8. Write a program to show rotation, scaling, and translation on an image.

```
import cv2

import numpy as np

# Load the image
image = cv2.imread('img.jpg')

# Define rotation angle (in degrees)
angle = 45

# Define scaling factors
scale_x = 1.5
scale_y = 1.5

# Define translation offsets
tx = 50
ty = 50

# Get image dimensions
height, width = image.shape[:2]

# Define rotation matrix
rotation_matrix = cv2.getRotationMatrix2D((width/2, height/2), angle, 1)

# Apply rotation
rotated_image = cv2.warpAffine(image, rotation_matrix, (width, height))

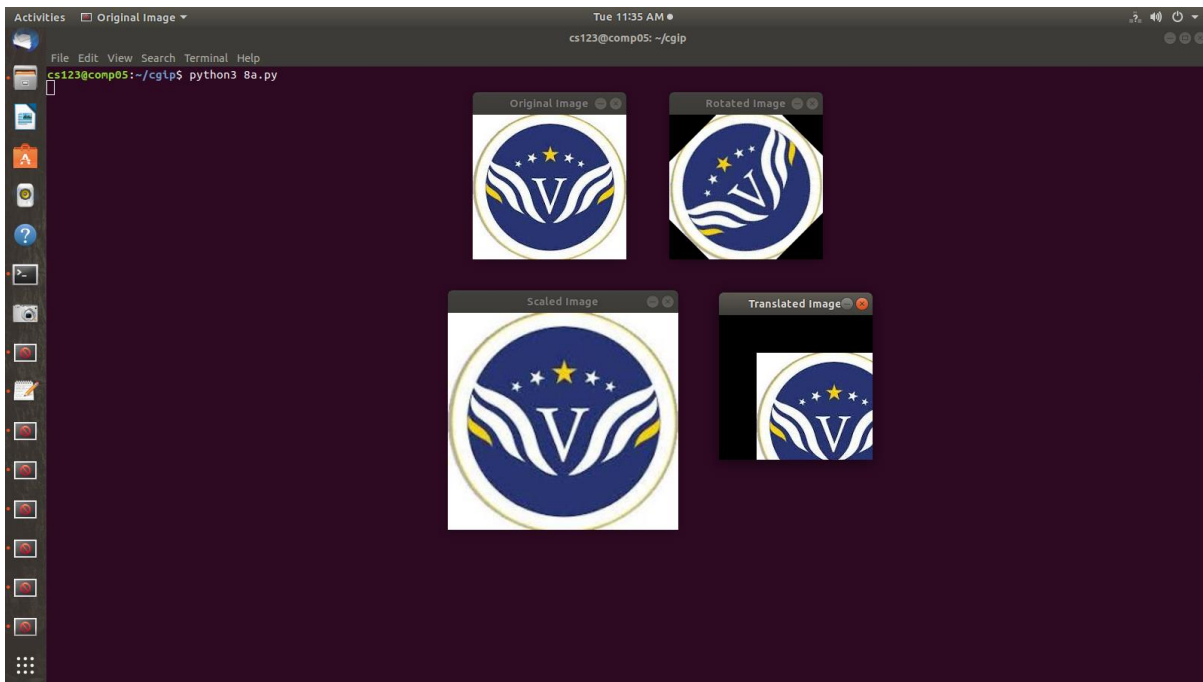
# Apply scaling
scaled_image = cv2.resize(image, None, fx=scale_x, fy=scale_y)

# Apply translation
translation_matrix = np.float32([[1, 0, tx], [0, 1, ty]])
translated_image = cv2.warpAffine(image, translation_matrix, (width, height))

# Display images
cv2.imshow('Original Image', image)
cv2.imshow('Rotated Image', rotated_image)
cv2.imshow('Scaled Image', scaled_image)
cv2.imshow('Translated Image', translated_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :



9. Read an image and extract and display low-level features such as edges, textures using filtering techniques

```
import cv2
```

```
import numpy as np
```

```
# Load the image
image_path = "3.jpeg" # Replace with the path to your image
img = cv2.imread(image_path)

# Convert the image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

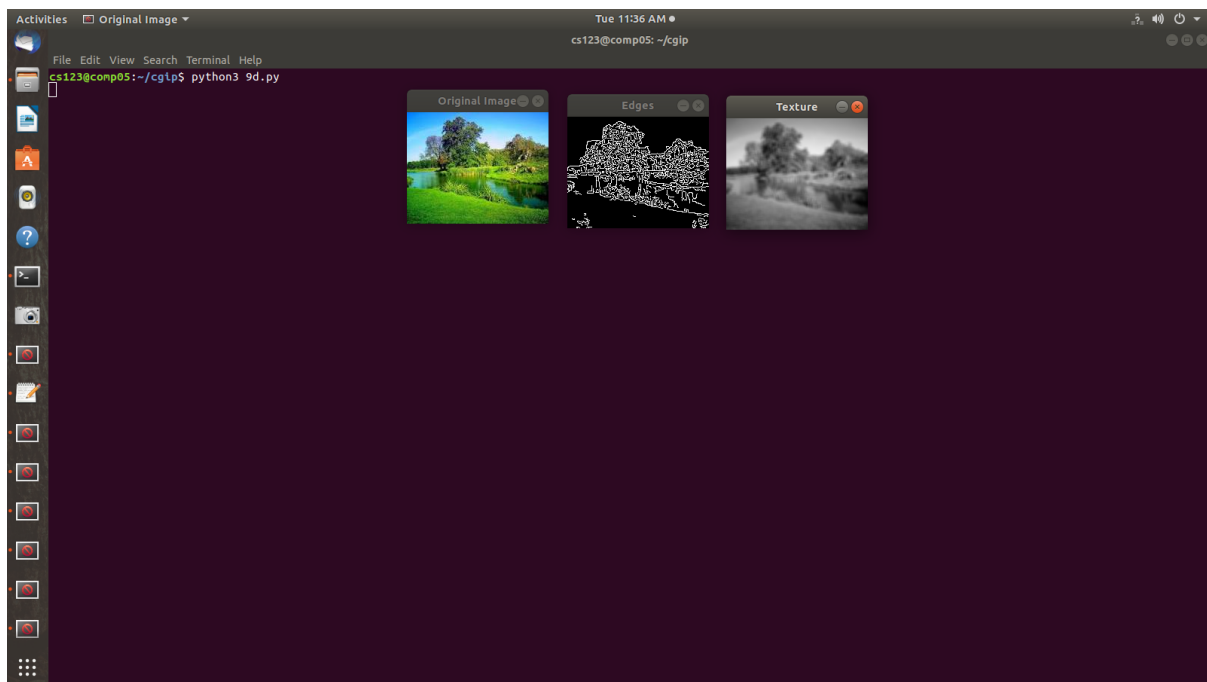
# Edge detection
edges = cv2.Canny(gray, 100, 200) # Use Canny edge detector

# Texture extraction
kernel = np.ones((5, 5), np.float32) / 25 # Define a 5x5 averaging kernel
texture = cv2.filter2D(gray, -1, kernel) # Apply the averaging filter for texture extraction

# Display the original image, edges, and texture
cv2.imshow("Original Image", img)
cv2.imshow("Edges", edges)
cv2.imshow("Texture", texture)

# Wait for a key press and then close all windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :



10. Write a program to blur and smoothing an image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read the image

img = cv2.imread("3.jpeg",cv2.IMREAD_GRAYSCALE)
image_array = np.array(img)
print(image_array)
def sharpen():
    return np.array([[1,1,1],[1,1,1],[1,1,1]])
def filtering(image, kernel):
    m, n = kernel.shape
    if (m == n):
        y, x = image.shape
        y = y - m + 1 # shape of image - shape of kernel + 1
        x = x - m + 1
        new_image = np.zeros((y,x))
        for i in range(y):
            for j in range(x):
                new_image[i][j] = np.sum(image[i:i+m, j:j+m]*kernel)
    return new_image
# Display the original and sharpened images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image_array,cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(filtering(image_array, sharpen()),cmap='gray')
plt.title("Blurred Image")
plt.axis("off")
plt.show()
```

Output :



11. Write a program to contour an image

```
import cv2

# Read the image
image = cv2.imread('face.jpeg')

# Convert the image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

_, im_th = cv2.threshold(gray_image, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Find contours in the threshold image
_, ctrs, hier = cv2.findContours(im_th.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

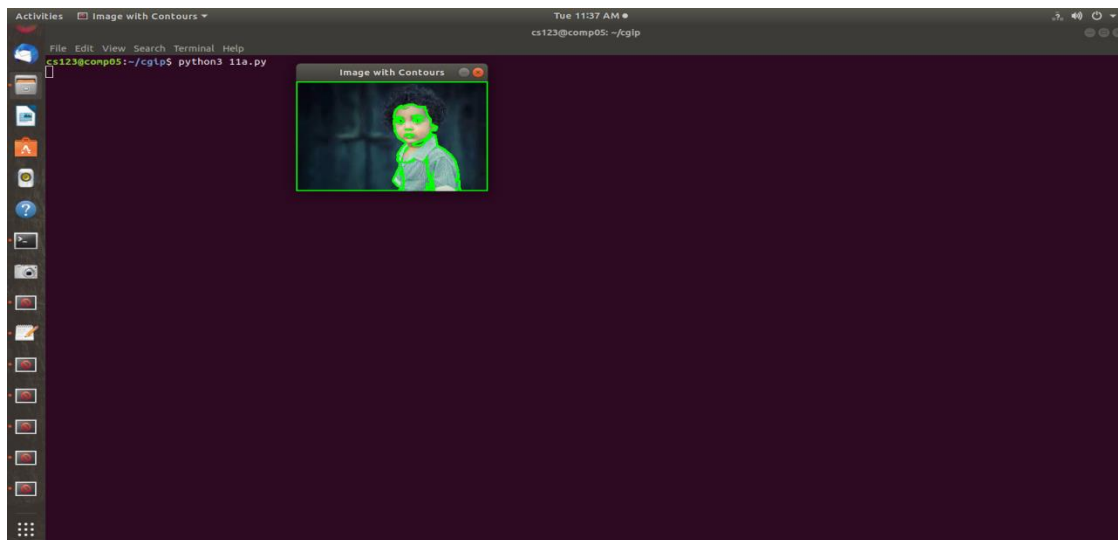
# Draw contours on the original image
contour_image = image.copy()

# Draw all contours with green color and thickness 2
cv2.drawContours(contour_image, ctrs, -1, (0, 255, 0), 2)

# Display the original image with contours
cv2.imshow("Image with Contours", contour_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



12: Write a program to detect a face/s in an image.

```
import cv2

# Load the pre-trained Haar Cascade classifier for face detection
```

```

face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_eye.xml')

# Read the input image (replace 'your_image.jpg' with the actual image path)
image_path = 'face.jpeg'
image = cv2.imread(image_path)

# Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces in the image
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)

# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Save or display the result
cv2.imwrite('detected_faces.jpg', image) # Save the result
cv2.imshow('Detected Faces', image) # Display the result
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Output:

